

Le Classi

Costruire i propri Data Types

pycache

È una cartella creata automaticamente quando si esegue codice Python. Funziona come una memoria cache che contiene file bytecode compilati (.pyc), permettendo a Python di eseguire il codice più velocemente nelle esecuzioni successive senza doverlo ricompilare. Non è necessario **versionare (cioè includerla nei sistemi di controllo versione come Git)** questa cartella perché il bytecode è specifico per la versione di Python usata e viene rigenerato automaticamente quando necessario. Per questo motivo viene solitamente inclusa nel file `.gitignore` per evitare di salvare file non necessari nel repository.

Objected Oriented Programming

La programmazione orientata agli oggetti (OOP) è un paradigma che permette di strutturare il codice in modo modulare ed efficiente attraverso l'uso delle **classi**.

Una classe rappresenta un concetto o un'entità, definendo le caratteristiche e i comportamenti comuni a più oggetti.

Comprendere le classi è essenziale per scrivere codice scalabile e manutenibile, ma la parte più complessa spesso non è tanto capire cosa sia una classe, quanto determinare quando e come utilizzarla correttamente.

Che cos'è una Classe

Una classe è un modello astratto che definisce le caratteristiche comuni di un insieme di oggetti.

In parole povere una classe è una generalizzazione di un'entità:

un modello astratto che definisce un insieme di caratteristiche e comportamenti comuni.

Le classi vengono usate per scrivere meno codice e renderlo più leggibile.

Esempio Pratico:

L'ITS deve sviluppare un gestionale per gestire il corso, qui si individuano i concetti che possono essere intesi come entità.

- gli studenti ad esempio possono essere generalizzati nella classe `studente` :
Questa classe non rappresenta un singolo studente, ma fornisce una struttura generica che descrive le caratteristiche e i comportamenti comuni a tutti gli studenti.

Attributi e metodi di una classe

Le classi di solito specificano gli attributi(dati) e metodi(funzioni).

1. Attributi:

Rappresentano le caratteristiche/proprietà della classe.

☰ Sono i rettangoli nei diagrammi ER.

2. Funzioni/Metodi:

Stabiliscono il comportamento della classe.

In altre parole **sono azioni che la classe può compiere o che possono essere compiute sulla classe.**

Quindi gli attributi della classe `Studente` , possono essere ad esempio:

- `nome`
- `cognome`
- `codice fiscale`
- `residenza` .

Mentre alcuni metodi potrebbero essere:

- `calcolo_media_voti()` :
per calcolare la media dei voti
- `calcolo_presenze()` :
Per determinare il numero di assenze e presenze.

☰ Esempio Pratico di un metodo:



Python

```
1 def calcola_media_voti(self, voti):  
2     return sum(voti) / len(voti) if voti else 0
```

Istanze di una classe

Un'istanza di una classe è un oggetto concreto creato a partire dalla classe stessa.

Possiamo immaginare questo concetto con un'analogia:

- **La classe è uno stampo per biscotti: definisce la forma generale, ma non è un biscotto in sé.**
- **Ogni biscotto prodotto con lo stampo è un'istanza della classe: anche se hanno la stessa forma, ogni biscotto è un oggetto distinto.**

Quindi con un solo stampo posso creare più biscotti (generalizzazione). Lo stampo non rappresenta nessun biscotto in particolare, ma se viene usato insieme alla pasta frolla produce un biscotto concreto. Ogni biscotto è diverso dall'altro (istanza).

Un'istanza è quindi un elemento unico che deriva da una classe, con valori specifici per i suoi attributi.

Creare un'istanza significa prendere la definizione generale della classe e assegnarle dati reali.

In termini pratici, un'istanza è una **realizzazione concreta** di una classe.

Tuttavia non bisogna confondere un'istanza con un attributo della classe.

Un'istanza è un oggetto completo, mentre un attributo è solo un dato che quell'oggetto contiene.

☰ Ad esempio:



Python

```
1 studente1 = Studente("Marco", "Pierleoni",  
    "MRCPLN98A01H501Z", "Roma")  
2 studente2 = Studente("Luca", "Bianchi", "LCABNC99B02F205T",  
    "Milano")
```

In questo caso, `studente1` e `studente2` **sono due istanze della classe** `Studente`. Il nome è un attributo, ma la stringa `"Marco"` è solo un valore assegnato a quell'attributo.

☰ Per fare chiarezza: Analogia con le istanze di un Plugin Audio

Le istanze di una classe in Python possono essere paragonate alle istanze di un plugin audio in una DAW: Immagina di utilizzare lo stesso identico plugin

sulla catena degli insert in una traccia, con gli stessi settaggi, in una DAW (Pro Tools, Reaper, Cubase, FL Studio, Ableton, ecc.). Sebbene si tratti dello stesso plugin (cioè della stessa classe), ogni istanza è indipendente e separata dalle altre. Ad esempio, consideriamo il compressore 1176 della UAD:



1176.png

- Il plugin UAD 1176 rappresenta una classe; definisce le caratteristiche e il comportamento del processamento della compressione sulla sorgente audio (traccia).
- Ogni volta che il plugin viene caricato in uno slot della traccia, viene creata una nuova istanza del plugin. Di conseguenza, anche se i parametri tra due istanze sono identici, esse sono comunque oggetti distinti con memoria separata. Lavorano e influenzano la CPU insieme, ma in modo indipendente.

Quindi, se modifichi un parametro su un'istanza del 1176, questa modifica non si riflette automaticamente sull'altra, proprio come se assegnassi un nuovo valore a un attributo di un'istanza di una classe in Python.

Sintassi delle istanze

Per creare una istanza di una classe in Python, bisogna seguire determinati passaggi:

1. Definire la classe:

- Creare una classe (`class NomeClasse`)
- Definire il metodo speciale `__init__` per inizializzare gli attributi

2. Definire **attributi** e **metodi**:

3. Creare un'istanza della classe:

- Dichiaro una variabile, in questo caso `studente1` , e gli si assegna una nuova istanza della classe

- Gli argomenti passati tra le parentesi servono per assegnare valori agli attributi (vengono anche chiamati **argomenti del costruttore**).
- In parole povere;



Python

```
1 istanza = NomeClasse()
```

≡ Esempio Pratico:



Python

```
1
2 class Studente:
3     def __init__(self, nome, cognome, codice_fiscale, citta):
4         self.nome = nome
5         self.cognome = cognome
6         self.codice_fiscale = codice_fiscale
7         self.citta = citta
8
9     def scheda(self):
10         return f"{self.nome} {self.cognome}, CF:
11         {self.codice_fiscale}, Città: {self.citta}"
12
13 # Creazione di istanze della classe Studente
14 studente1 = Studente("Marco", "Pierleoni",
15                       "MRCPLN98A01H501Z", "Roma")
16 studente2 = Studente("Luca", "Bianchi", "LCABNC99B02F205T",
17                       "Milano")
18
19 # Utilizzo di un metodo della classe
20 print(studente1.scheda())
21
22 # Output: Marco Pierleoni, CF: MRCPLN98A01H501Z, Città: Roma
```

In questo esempio:

- Il **costruttore** `__init__` inizializza gli attributi
- Quando crei `studente1`, gli passi i valori che vengono assegnati agli attributi della classe

- Il metodo `scheda()` permette di ottenere una rappresentazione testuale dell'istanza

Come fa Python a determinare se due istanze sono lo stesso oggetto?



Python

```
1 studente:studente = studente("Marco", "Pierleoni")
2 studente:studente = studente("Marco", "Pierleoni")
```

Python confronta gli indirizzi di memoria delle due istanze. Anche se `studente1` e `studente2` hanno gli stessi valori negli attributi, sono due oggetti distinti perché occupano posizioni diverse in memoria.

Quindi per ricapitolare:

Le classi in Python sono modelli che definiscono come devono essere strutturati gli oggetti.

Da un punto di vista pratico, la classe è come uno stampo per creare oggetti che hanno attributi (caratteristiche) e metodi (azioni). Ogni volta che crei un'istanza di una classe, stai creando un oggetto unico, proprio come ogni plugin in una DAW, che, pur essendo basato sulla stessa "classe", funziona in modo indipendente con proprie impostazioni e memoria.

In altre parole, una classe è un template per generare istanze distinte che condividono la stessa struttura ma possono avere valori diversi.

Riprendendo l'esempio del gestionale dell'ITS:

Oltre alla classe `Studente` ci possono essere altre classi come:

- `Insegnanti`
- `Amministrazione`
- `Dispositivo`
- `Aula`
- `Corso`
- `Sede` (se ci sono più sedi)
- etc.

In questo caso, le classi rappresentano **oggetti tangibili** all'interno del sistema. Tuttavia, le classi in Python possono anche rappresentare **concetti astratti**, come nel caso dei **Design Pattern**:

sono schemi di programmazione che aiutano a scrivere codice più efficiente e riutilizzabile, e sono fondamentali per il design del software

(il libro di riferimento è *Design Patterns: Elements of Reusable Object-Oriented Software* pubblicato negli anni '80).

Questi schemi possono essere implementati in qualsiasi linguaggio, e **Python li supporta nativamente**.

Ad esempio, il Design Pattern *Strategy*:

è uno schema che definisce una famiglia di algoritmi, incapsulando ciascuno di essi in una classe separata.

Queste classi rappresentano **concetti astratti** che permettono di scegliere dinamicamente l'algoritmo da usare, a seconda della situazione. Per ora, tuttavia, ci concentreremo su concetti più **concreti**.

Quando si confrontano due istanze, bisogna tenere a mente che le classi in Python sono come le tabelle di un database, dove non possono esserci due chiavi primarie uguali.

Sebbene in Python di default due istanze con lo stesso valore vengano considerate come "uguali" (perché confrontano i loro **attributi**), a livello di implementazione bisogna fare attenzione, poiché si potrebbe trattare di due oggetti distinti e separati.

Per capire meglio prendiamo l'esempio fatto sopra con le due istanze di `'studente1'` e `'studente2'`:

In questo caso, `studente1` e `studente2` sono due istanze della classe `Studente`. Sebbene abbiano un attributo simile (il nome e cognome), sono comunque oggetti distinti, con valori diversi per gli altri attributi (come il codice fiscale e la città).

Questo dimostra che anche se le istanze appartengono alla stessa classe, i loro attributi possono differire e, di conseguenza, sono trattate come oggetti separati.

Un esempio pratico di come i Design Pattern vengano utilizzati in Python è su Google Maps. Se vogliamo implementare più algoritmi che fanno la stessa cosa, come ad esempio calcolare il percorso più breve, possiamo usare il *Strategy Pattern*, dove ogni "strategia" è rappresentata da un algoritmo differente.

Per approfondire i Design Pattern, oltre al libro, consiglio il sito [Refactoring Guru](#), che offre una panoramica dettagliata dei vari schemi di progettazione.

Ereditarietà delle classi in Python

L'ereditarietà è un meccanismo che permette a una classe (detta classe derivata o sottoclasse) di ereditare attributi e metodi da un'altra classe

(detta classe base o superclasse).

Questo consente di riutilizzare il codice ed evitare duplicazioni, migliorando la **manutenibilità** e la **leggibilità** del programma.

Per spiegare al meglio il concetto di ereditarietà delle classi in Python riprendiamo il solito esempio del gestionale dell'ITS:

Abbiamo definito 4 classi per l'ITS, tutte queste classi, se ci pensiamo bene, hanno una cosa in comune:

sono tutte persone, anche se ogni classe fa una cosa diversa.



Ereditarietà delle classi.png

Questa immagine rappresenta la gerarchia di classi utilizzando il concetto di ereditarietà nella programmazione OOP:

1. Classe "Persona" (superclasse)

- **È la classe più generica della gerarchia.**
- Introduce due attributi: **età e altezza** (indicati come **Nuovo** in verde perché sono definiti per la prima volta).

2. Classe "Studente" (sottoclasse di "Persona")

- **Eredita gli attributi età e altezza dalla classe Persona.**
- Introduce due nuovi attributi: **MatrNr (numero di matricola) e Università** (**Nuovo** in verde).

1. Classe "Studente di scambio" (sottoclasse di "Studente")

- Eredita tutti gli attributi delle classi superiori:
 - **età, altezza** (da Persona)
 - **MatrNr, Università** (da Studente)

- Aggiunge un nuovo attributo: **Università di origine (Nuovo in verde).**

Cosa mostra l'immagine?

Ogni classe eredita gli attributi dalla classe superiore, evitando la ripetizione del codice.

La gerarchia è costruita con il principio "**is-a**":

- **Uno** `Studiante di scambio` **è anche uno** `Studiante` **ed è anche una** `Persona`.
- **Più si scende nella gerarchia, più le classi diventano specifiche**.

Queste 4 classi che possono essere raggruppate in una superclasse `Persona` perché hanno attributi in comune (ad esempio `Nome`, `Cognome`, `Codice Fiscale`, `Residenza` etc.).

Quindi andando a creare la classe `Persona` **si va a formare una gerarchia di classi padri e classi figli**.

🕒 **In questo caso la classe** `Persona` **, diventa la classe padre di** `Studiante`, `Insegnante` **ed etc.**

Quindi `Persona` ha la priorità più alta nella gerarchia delle classi

Questa gerarchia padre-figlio è utile per non scrivere tutti gli attributi e i metodi per ogni classe.

Così facendo scrivo meno codice ogni volta per ogni classe che si riferiscono a persone, inoltre la possibilità di sbagliare è più bassa.

Prendendo in riferimento l'immagine sopra, possiamo notare come le frecce indicano che il flusso vada dal basso verso l'alto, questo perché non tutte le persone sono studenti o studenti di scambio, ma tutti gli studenti sono persone e anche tutti gli studenti di scambio sono persone.

Per comprendere meglio questo concetto:

`Studiante` **eredita la classe** `Persona` **, quindi lo studente è una persona ma non il contrario e non può andare nel senso opposto perché posso dire che** `A` **(cioè la superclasse** `Persona` **) eredita** `B` **(la sottoclasse** `Studiante` **) ma** `B` **non eredita** `A` **, tuttavia** `C` **(la sottoclasse** `Studiante di scambio` **) può ereditare sia** `A` **che** `B`.

Come possiamo notare questa formula dell'ereditarietà delle classi segue il sillogismo aristotelico:

1. Tutti gli uomini sono mortali
2. Socrate è un uomo
3. Quindi Socrate è mortale

Nel nostro caso, l'analogia si traduce così:

1. **Premessa maggiore:**

"Tutti gli studenti sono Persone "

(Ogni istanza della classe `Studente` eredita le proprietà/metodi della superclasse `Persona`).

Quindi è come dire che le istanze di `Studente` sono parte delle istanze di `Persona`

2. **Premessa minore:**

"Non tutte le Persone sono Studenti "

(Ciò significa che `Studente` è un **sottoinsieme proprio** della classe `Persona` , quindi l'inverso non è valido.)

3. **Conclusione:**

"L'insieme di `Studente` è un sottoinsieme proprio di `Persona` "

(La gerarchia diventa chiara: `Persona` → `Studente`).

✓ Se `B` eredita da `A` , allora `B` è un(**is-a**) `A` .

Questo perché io posso ereditare padre → figlio ma poi il figlio a sua volta può diventare padre, quindi $A \rightarrow B \rightarrow C$.

Quindi, in sostanza:

`B` eredita `A` ma `C` eredita sia `A` che `B` .

≡ **Analogia tra Programmazione OOP e DAW:** >

L'ereditarietà in programmazione può sembrare astratta, ma possiamo comprenderla meglio attraverso un'analogia con il mondo dell'audio.

In una DAW (come Reaper, Pro Tools, Cubase, etc.), organizziamo le tracce in **bus** per gestirle più facilmente, proprio come in OOP creiamo **classi e sottoclassi** per evitare ripetizioni e migliorare la gestione del codice.

Supponiamo di avere `n` tracce di batteria:

- Kick In
- Kick Out
- Snare Up
- Snare Bottom
- Tom1, Tom2, Tom3, Tom4, Floor Tom

- Overhead Left, Overhead Right
- Drum Room Left, Drum Room Right

Ogni singola traccia può essere vista come un **oggetto** (istanza di una classe), con le sue caratteristiche (attributi) e azioni applicabili (metodi):

- **Attributi:**
caratteristiche della traccia (es. `KickIn` ha un certo livello di volume, un certo colore sonoro, ecc.).
- **Metodi:**
azioni applicabili sulla traccia (EQ, compressione, riverbero, regolazione del pan, regolazione del volume di uscita del canale).

Per comodità e per velocizzare il flusso di lavoro, queste tracce vengono raggruppate in **bus group tracks** (tracce di gruppo). Ad esempio:

- Le tracce del **Kick** (`Kick In` + `Kick Out`) vengono raggruppate in una traccia bus `Kick` .
- Le tracce dello **Snare** (`Snare Up` + `Snare Bottom`) vengono raggruppate in un bus `Snare` .
- Le tracce dei **Tom** (`Tom1` , `Tom2` , etc.) vanno in un bus `Toms` .
- Tutte queste **bus group tracks** vengono infine raggruppate in un'unica traccia di gruppo chiamata `Drum Kit` .

Da qui possiamo fare un parallelismo con la Programmazione OOP: Questa struttura è simile alla gerarchia padre-figlio delle classi in programmazione orientata agli oggetti.

- **Non tutte le tracce sono Kick o Snare**, ma tutte fanno parte della batteria.
- **Non tutte le persone sono studenti, e non tutti gli studenti sono studenti di scambio**, ma ogni studente di scambio è sia uno studente che una persona.

Esattamente come in programmazione classi specifiche (come `studenti di scambio`) possono ereditare attributi e metodi da classi più generali (`Studente` e `Persona`).

Più si sale nella gerarchia, più il bus diventa generico, proprio come la classe `Persona` che rappresenta il livello più alto della gerarchia.

Quindi i **le singole tracce di batteria:**

Rappresentano gli **oggetti** (istanze) della classe e i loro **attributi** e **metodi**.

I **Bus Group (Kick, Snare, Toms, ecc.):**

Rappresentano le **sottoclassi**, ovvero raggruppano più tracce simili in un'unica entità con caratteristiche comuni.

Esattamente come la classe `studente di scambio` eredita attributi e metodi della classe `Studente`.

Il **Bus Drum Kit (Superclasse)**:

Il Bus Drum Kit unisce Kick, Snare, Overheads, ecc., proprio come la classe `Persona` è la superclasse di `Studente`.

Ovviamente la gerarchia nella programmazione non è intesa come flusso di segnali come in una DAW, ma un flusso di astrazione.

Tuttavia la logica della categorizzazione è simile.

Esattamente come nella gestione del suono, la gerarchia delle classi permette di organizzare il codice in modo più efficiente e modulare.

✓ **Vantaggio**

Se voglio applicare una compressione comune a tutte le tracce di batteria, lo faccio sul **bus Drum Kit**, invece di settare manualmente ogni singola traccia → lo stesso principio vale nella programmazione: eredito attributi e metodi dalla superclasse per evitare ripetizioni inutili.

≡ **Parallelo completo tra DAW e programmazione a oggetti**

Daw	Programmazione (OOP)
Tracce singole (KickIn, SnareUp, Tom1,etc.)	Oggetti/istanze con attributi e metodi
Moduli di canale (EQ, Send, Pan, Fader,etc.)	Metodi (azioni eseguibili sulla traccia)
Bus Kick, Snare, Toms, etc.	Sottoclassi (ereditarietà da una classe più generica)
Bus Drum Kit	Superclasse (classe base da cui tutte le altre ereditano)

Quando si creano gerarchie di classi, si definiscono nuovi tipi di dati. Ad esempio, la classe `Studente` eredita da `Persona`, il che significa che uno studente è **una**

persona (relazione is-a). Più si scende nella gerarchia, più le classi diventano specifiche.

Se osserviamo l'immagine, la classe `Studente di scambio` (*Exchange Student*) è collegata con una freccia alla classe `Studente`. Questo perché **non tutti gli studenti sono studenti di scambio**, così come **non tutte le persone sono studenti**, ma **tutti gli studenti sono persone**.

② Principio "is-a"

Il principio *is-a* nella gerarchia dell'ereditarietà delle classi in Python rappresenta una relazione in cui una classe derivata è un tipo di oggetto della classe base. In altre parole, una classe derivata "è una" classe base, estendendo o specializzando il comportamento della classe da cui eredita.

In parole povere questo principio riguarda specificatamente la relazione in cui la sottoclasse "è una" istanza della classe base, e può estenderla o specializzarla .

≡ Esempio di come si applica il principio is-a:

```
Python
```

```
1 class Animale:
2     def fare_suono(self):
3         pass
4
5 class Cane(Animale):
6     def fare_suono(self):
7         return "Abbaia"
8
9 class Gatto(Animale):
10    def fare_suono(self):
11        return "Miao"
12
13 # Uso del principio is-a
14 animale = Cane()
15 print(isinstance(animale, Animale)) # True, perché Cane è
16 un tipo di Animale
```

In questo esempio:

- **Cane** e **Gatto** sono classi derivate da **Animale**.
- Un oggetto di tipo **Cane** o **Gatto** è *una* classe **Animale**, e quindi può essere trattato come un'istanza di **Animale**.

Il principio *is-a* è fondamentale per modellare gerarchie di classi in modo che oggetti di classi derivate possano essere utilizzati ovunque siano richiesti oggetti della classe base, sfruttando il polimorfismo.

Come procedere nella progettazione?

Ci sono diversi modi per strutturare una gerarchia di classi, ma è fondamentale seguire alcuni principi:

1. Non scrivere subito codice:

Prima di iniziare a programmare, è essenziale progettare su carta, specialmente quando si lavora su sistemi complessi.

Bisogna:

- **Identificare le classi**
- **Definire gli attributi e i metodi**
- **Stabilire le relazioni di ereditarietà**

2. Scegliere un approccio di progettazione:

Si può procedere in due modi:

- **Dalla generalizzazione alla specializzazione:**
Partire da concetti generali e poi raffinare le classi più specifiche.
- **Dalla specializzazione alla generalizzazione:**
Partire da elementi concreti e raggrupparli in categorie più generali.

I modelli in programmazione

Un modello è un'astrazione della realtà .

Esistono molti tipi di modelli, ognuno dei quali descrive un aspetto specifico del mondo che ci circonda. Ad esempio, la **legge di gravitazione di Newton** è un modello che approssima il comportamento della gravità, senza rappresentare ogni dettaglio della realtà.

Anche in programmazione i modelli possono essere **molto specifici** o **più generici**.

Un modello troppo dettagliato potrebbe richiedere una gestione complessa dei dati, rallentando il sistema.

D'altra parte, un modello troppo generico potrebbe risultare poco utile per risolvere problemi concreti.

La scelta del livello di astrazione dipende quindi dal programmatore e dalle esigenze del sistema che si sta sviluppando.

Implementare le classi in Python

In Python, le classi permettono di creare strutture dati personalizzate e oggetti con caratteristiche specifiche.

Una classe rappresenta un modello per la creazione di oggetti, ciascuno con attributi e metodi propri.

Definizione di una Classe e il Costruttore

`__init__()`

Per definire una classe in Python, si utilizza la parola chiave `class`.

Gli attributi di una classe vengono generalmente assegnati attraverso il **costruttore**, un metodo speciale chiamato `__init__()`.

Questo metodo viene eseguito automaticamente ogni volta che viene creata una nuova istanza della classe.

🔗 Il costruttore `__init__` ✓

Il metodo `__init__` non è obbligatorio ma è fortemente consigliato per inizializzare degli attributi specifici per ogni istanza della classe.

Quindi il costruttore è utile per inizializzare gli attributi dell'oggetto, assegnando valori specifici quando viene creata un'istanza.

Senza di esso si può comunque creare una classe e istanziare gli oggetti, ma si deve farlo manualmente:



Python

```
1 class Persona:
2     pass # Classe vuota
3
4 # Creiamo un'istanza
5 persona1 = Persona()
```

```

6
7  # Aggiungiamo attributi manualmente
8  persona1.nome = "Mario"
9  persona1.cognome = "Rossi"
10  persona1.eta = 30
11
12  print(persona1.nome)  # Mario

```

Tuttavia ogni volta che si crea una nuova istanza devi assegnare gli attributi manualmente, il che è scomodo e poco pratico.

Mentre con `__init__`, gli attributi vengono assegnati automaticamente:



Python

```

1  class Persona:
2      def __init__(self, nome, cognome, eta):
3          self.nome = nome
4          self.cognome = cognome
5          self.eta = eta
6
7  # Creiamo un'istanza
8  persona1 = Persona("Mario", "Rossi", 30)
9
10  print(persona1.nome)  # Mario

```

✓ Vantaggi

Ogni istanza viene creata con i suoi dati senza bisogno di assegnarli manualmente.

Il codice è più chiaro e organizzato.

Quindi `__init__` **serve solo per inizializzare gli attributi di un'istanza** ma non i metodi della classe.

Difatti si può definire metodi senza usare `__init__`:



Python

```

1  class Persona:
2      def saluta(self):
3          return "Ciao, piacere di conoscerti!"
4
5  p = Persona()

```



```
6 print(p.saluta()) # Output: Ciao, piacere di conoscerti!
```

✏ Anche se `__init__()` non è obbligatorio per creare metodi, spesso è usato insieme ad essi per lavorare con gli attributi della classe.

☰ In conclusione

- `__init__()` non è obbligatorio, ma è utile per inizializzare gli attributi di un'istanza.
- Non serve per creare funzioni o metodi, ma solo per assegnare gli attributi di un oggetto.
- Una classe può avere altri metodi oltre a `__init__()`, indipendenti da esso.

Ecco un esempio di definizione della classe `Person`, che rappresenta una persona con tre attributi: `nome`, `cognome` ed `eta`.



Python

```
1 class Person:
2     def __init__(self, nome, cognome, eta):
3         self.nome = nome
4         self.cognome = cognome
5         self.eta = eta
```

- `self`:
rappresenta l'istanza attuale della classe e permette di accedere agli attributi e ai metodi dell'oggetto.

Self in Python

Il `self` è un riferimento all'istanza della classe ed è utilizzato per accedere agli attributi e ai metodi dell'oggetto.

Grazie a `self`, ogni istanza mantiene i propri dati separati dalle altre, evitando sovrapposizioni.

È quindi indispensabile nei metodi di una classe quando si lavora con gli attributi dell'istanza.

Dove si usa `self` ?

1. Nel costruttore `__init__`

Quando un oggetto viene creato, `self` **permette di assegnare i valori agli attributi dell'istanza.**



Python

```
1 class Persona:
2     def __init__(self, nome, cognome):
3         self.nome = nome # ✅ Necessario per salvare il valore
        nell'istanza
4         self.cognome = cognome
5
```

2. Nei metodi d'istanza:

`Self` **permette di accedere agli attributi dell'oggetto anche dentro altri metodi della classe.**



Python

```
1 class Persona:
2     def __init__(self, nome):
3         self.nome = nome # ✅ Memorizza il nome nell'oggetto
4
5     def saluta(self):
6         return f"Ciao, mi chiamo {self.nome}!" # ✅ Usa self per
        leggere l'attributo
7
8     personal = Persona("Luca")
9     print(personal.saluta()) # ✅ Ciao, mi chiamo Luca!
10
```

`Self` **è solo una convenzione**

Python non obbliga a usare proprio la parola `self`, ma è una convenzione universale. **Rinominare `self` con un altro nome è considerato una cattiva pratica, perché rende il codice più difficile da leggere e capire.**



Python

```
1 class Persona:
```

```
2     def __init__(istanza, nome): # Tecnicamente valido, ma  
    sconsigliato  
3         istanza.nome = nome
```

Il `self` è importante perché senza di esso gli attributi non vengono salvati in RAM per l'istanza:

- Con `self` l'attributo viene memorizzato nell'oggetto e rimane disponibile finché esiste l'istanza
- Senza `self` il valore esiste solo dentro la funzione e viene eliminato quando la funzione termina.



Python

```
1 class Persona:  
2     def __init__(self, nome):  
3         nome = nome # Non viene salvato nell'istanza!  
4  
5     personal = Persona("Luca")  
6     print(personal.nome) # Errore! L'attributo non esiste
```

≡ In Conclusione: l'importanza del self

- `Self` è un parametro che serve per accedere agli attributi e ai metodi di un'istanza della classe.
- Permette di salvare gli attributi nell'istanza dell'oggetto, mantenendoli in memoria finché l'istanza esiste.
- È obbligatorio nei metodi d'istanza per distinguere i dati dell'oggetto corrente da quelli di altre istanze.
- Non è una parola riservata, ma è una convenzione fortemente consigliata.

Quindi, grazie a `Self` ogni oggetto viene reso unico e separato dagli altri.

- `nome`, `cognome` ed `eta`:
sono i parametri passati quando si crea una nuova istanza della classe e vengono assegnati agli attributi dell'oggetto.

Quando viene creato un nuovo oggetto, Python alloca memoria per l'istanza e i suoi attributi.

Ogni istanza è indipendente dalle altre, occupando uno spazio di memoria separato.

Creazione di Oggetti

Per creare un'istanza della classe `Person`, basta chiamare il costruttore passando i valori richiesti:



Python

```
1  persona1 = Person("Mario", "Rossi", 30)
2  persona2 = Person("Flavio", "Rossi", 31)
3  persona3 = Person("Flavia", "Asti", 29)
4  persona4 = Person("Flavio", "Adducci", 31)
```

Ogni variabile (`persona1`, `persona2`, ecc.) rappresenta un'istanza distinta della classe `Person` con dati specifici.

Gestione degli Attributi con Getter e Setter

In Python, gli attributi di un oggetto sono generalmente accessibili direttamente, ma è buona pratica utilizzare metodi specifici per leggere e modificare i valori.

Questo permette di applicare controlli e garantire un accesso sicuro ai dati. Esistono 2 metodi principali per gestire gli attributi:

1. Getter:

permettono di ottenere il valore di un attributo.

In altre parole è un metodo read-only, restituisce il valore dell'attributo e permette di leggerlo senza modificarlo.

2. Setter:

consentono di modificare i valori degli attributi in modo controllato.

In parole povere permette di modificare il valore di un attributo, applicando eventualmente controlli o validazioni.

Implementazione di Getter e Setter

Un approccio tradizionale per gestire gli attributi di una classe è definire esplicitamente i metodi `get` e `set`.

Esempio di getter e setter per l'attributo `nome`:



Python

```
1 class Person:
2     def __init__(self, nome, cognome, eta):
3         self.nome = nome
4         self.cognome = cognome
5         self.eta = eta
6
7     def get_nome(self):
8         return self.nome
9
10    def set_nome(self, nome):
11        self.nome = nome
```

Questo approccio permette di gestire l'accesso agli attributi e aggiungere logiche di validazione in futuro.

Accessibilità degli Attributi: Pubblici e Privati

In Python, non esistono modificatori di accesso espliciti come in altri linguaggi (ad esempio `public`, `private` in Java).

Tuttavia, per convenzione, **un attributo può essere reso "privato" utilizzando un underscore (`_`) o due underscore (`__`) prima del nome dell'attributo.**

Esempio:



Python

```
1 class Person:
2     def __init__(self, nome, cognome, eta):
3         self.__nome = nome # Attributo privato
```

L'uso di `__` (doppio underscore) attiva il name mangling, che modifica il nome dell'attributo internamente per rendere più difficile l'accesso diretto dall'esterno della classe. Tuttavia, non è una protezione assoluta.

Utilizzo dei Decoratori `@property` per Getter e Setter

Un'alternativa più elegante per gestire l'accesso agli attributi è **l'uso dei decorator `@property` e `@nome.setter`, che permettono di definire getter e setter in modo più intuitivo :**



Python

```

1  class Person:
2      def __init__(self, nome, cognome, eta):
3          self.__nome = nome
4          self.__cognome = cognome
5          self.__eta = eta
6
7      @property
8      def nome(self):
9          return self.__nome
10
11     @nome.setter
12     def nome(self, nome):
13         if len(nome) > 0:
14             self.__nome = nome
15         else:
16             raise ValueError("Il nome non può essere vuoto")

```

Questo approccio:

- Il metodo `nome()` con il decoratore `@property` agisce come un getter: **permette di leggere l'attributo come se fosse pubblico, ma senza modificarlo direttamente.**
- Il metodo `nome()` con il decoratore `@nome.setter` agisce come un setter: **permette di applicare controlli sulla modifica del valore.**

👁 **Ovviamente per ogni attributo (`nome` , `cognome` , `eta`) bisogna usare il nome dell'attributo stesso nel decoratore (`@nome.setter` , `@cognome.setter` , `@eta.setter`)**

☰ Esempio: >



Python

```

1  class Person:
2      def __init__(self, nome, cognome, eta):
3          self.__cognome = cognome
4          self.__eta = eta
5
6      # Getter e Setter per nome
7      @property
8      def nome(self):
9          return self.__nome
10

```

```

11     @nome.setter
12     def nome(self, nuovo_nome):
13         if len(nuovo_nome) > 0:
14             self.__nome = nuovo_nome
15         else:
16             raise ValueError("Il nome non può essere vuoto")
17
18     # Getter e Setter per cognome
19     @property
20     def cognome(self):
21         return self.__cognome
22
23     @cognome.setter
24     def cognome(self, nuovo_cognome):
25         if len(nuovo_cognome) > 0:
26             self.__cognome = nuovo_cognome
27         else:
28             raise ValueError("Il cognome non può essere
29 vuoto")
30
31     # Getter e Setter per eta
32     @property
33     def eta(self):
34         return self.__eta
35
36     @eta.setter
37     def eta(self, nuova_eta):
38         if nuova_eta > 0:
39             self.__eta = nuova_eta
40         else:
41             raise ValueError("L'età deve essere positiva")

```

✓ Vantaggi dell'uso di Getter e Setter con @property

- **Incapsulamento:** Permette di nascondere l'implementazione interna e di esporre solo ciò che è necessario.
- **Controllo e validazione:** È possibile inserire controlli sui dati prima di modificarli.
- **Interfaccia coerente:** Gli attributi possono essere letti e modificati come se fossero pubblici, ma con un maggiore controllo.

- **Compatibilità futura:** Se in futuro si vorrà modificare l'accesso a un attributo (ad esempio rendendolo calcolato dinamicamente), non sarà necessario cambiare il codice esistente che usa la classe.

In conclusione, L'uso di getter e setter in Python dipende dalle necessità del progetto.

Se non è necessario applicare controlli sugli attributi, si può accedere direttamente a essi. Tuttavia, per un codice più robusto e manutenibile, è consigliato l'uso di `@property` per una gestione più sicura e controllata degli attributi.

☰ In sintesi

- Gli **attributi pubblici** sono accessibili direttamente, ma potrebbero causare problemi di sicurezza e inconsistenza.
- Gli **attributi privati** (con `__`) riducono il rischio di modifiche accidentali, ma non sono completamente inaccessibili.
- I **getter e setter con** `@property` sono la soluzione più pulita per mantenere il codice leggibile e sicuro.

Ereditarietà: Creare Classi Derivate

L'ereditarietà consente di creare una nuova classe basata su un'altra esistente, evitando di riscrivere codice già definito.

Una classe derivata eredita gli attributi e i metodi della classe base, ma può anche definirne di nuovi o sovrascrivere quelli esistenti.

Esempio di una classe `Dipendente` che estende `Persona`, aggiungendo l'attributo `stipendio`:



Python

```
1 class Persona:
2     def __init__(self, nome, cognome, eta):
3         self.nome = nome
4         self.cognome = cognome
5         self.eta = eta
6
7 class Dipendente(Persona):
8     def __init__(self, nome, cognome, eta, stipendio):
```



```

9         super().__init__(nome, cognome, eta) # Richiama il
           costruttore della classe base
10         self.stipendio = stipendio # Nuovo attributo specifico
           della classe derivata
11
12     def get_stipendio(self):
13         return self.stipendio

```

In questo esempio:

- La classe `Dipendente` **eredita** da `Persona`, quindi può usare i suoi attributi (`nome`, `cognome`, `eta`).
- Il metodo `super().__init__(nome, cognome, eta)` chiama il costruttore della classe `Persona`, evitando di riscrivere lo stesso codice.
- **Aggiungiamo un nuovo attributo** `stipendio`, che è specifico della classe `Dipendente`.

Quindi l'uso di `super().__init__()` **garantisce che il costruttore della classe base (classe padre) venga chiamato correttamente, evitando ripetizioni di codice**.

Questo è utile quando si eredita da un'altra classe e si vuole **riutilizzare** il codice del costruttore della classe base senza riscriverlo.

✓ Perché usare `super().__init__()` ?

- **Evita ripetizioni di codice:**
Se la classe `Persona` ha molti attributi, non serve riscriverli in `Dipendente`.
- **Permette di estendere la classe padre** senza modificarne il codice.
- **Gestisce automaticamente l'ereditarietà multipla** in modo pulito.

Protezione degli Attributi Sensibili con Eccezioni

In alcuni casi, è utile impedire l'accesso o la modifica di determinati attributi, come una password.

Si può fare lanciando un'eccezione se si tenta di accedere a un valore sensibile:



Python

```

1 class User:
2     def __init__(self, username, password):

```

```

3         self.username = username
4         self.__password = password
5
6     def get_password(self):
7         raise ValueError("Non puoi accedere alla password")

```

In questo modo, chi tenta di leggere `get_password()` otterrà un'eccezione, proteggendo il dato.

☰ Conclusione

Riassumendo:

- `**__init__()` :
è il metodo costruttore usato per inizializzare un oggetto.
- `**self` :
permette di riferirsi all'istanza corrente della classe.
- Getter e setter permettono di controllare l'accesso agli attributi.
- L'uso di `_` e `__` aiuta a definire attributi "privati".
- I decorator `@property` offrono un modo più elegante per gestire gli attributi.
- L'ereditarietà permette di riutilizzare e estendere il codice esistente.
- Le eccezioni possono essere usate per proteggere dati sensibili.

L'uso corretto delle classi rende il codice più organizzato, leggibile e manutenibile, consentendo di modellare in modo efficace problemi complessi con la programmazione a oggetti in Python.

Attributi di classe

Gli **attributi di classe** sono attributi **globali**:

non appartengono ai singoli oggetti (istanze), ma sono condivisi da tutti gli oggetti della stessa classe.

Quindi in sostanza: sono condivisi tra tutte le istanze di una classe, a differenza degli attributi di istanza (definiti con `self` in `__init__()`), e quindi gli attributi di classe sono legati alla classe stessa e non ai singoli oggetti.

✓ A cosa servono?

- **Condividere dati globali:** Utili per memorizzare informazioni comuni a tutte le istanze (es. un contatore di oggetti creati).
- **Risparmiare memoria:** Poiché il valore è unico per la classe, non viene duplicato in ogni oggetto.

Per fare un esempio più concreto: mettiamo di avere ho 10 oggetti della classe `Person` , **un attributo di classe sarà lo stesso per tutti gli oggetti, perché è associato direttamente alla classe e non ai singoli oggetti.**

Esempio Pratico:



Python

```
1 class Persona:
2     # Attributo di classe (condiviso)
3     conteggio_personone = 0
4
5     def __init__(self, nome: str):
6         self.nome = nome # Attributo di istanza (unico per
7         self.aggiorna_conteggio()
8
9     @classmethod
10    def aggiorna_conteggio(cls) -> None:
11        cls.conteggio_personone += 1
12
13    # Creazione di istanze
14    alice = Persona("Alice W.")
15    bob = Persona("Bob M.")
16
17    print(f"Persone create: {Persona.conteggio_personone}") # Output:
18    2
```

Spiegazione passo-passo:

- `personCount` è un **attributo di classe**: esiste **una sola copia** di `personCount` , condivisa da tutti gli oggetti `Person` .
- Ogni volta che creiamo un nuovo oggetto (`Person("Alice W.")` , `Person("Bob M.")`), il metodo `update()` viene chiamato e incrementa `personCount` di 1.
- Alla fine, `Person.personCount` vale **2**.

- `cls` : è il modo corretto per accedere agli attributi della classe.

Distinzione tra `@classmethod` e `cls`

- `@classmethod` è un decoratore che segnala che il metodo successivo (ovvero `update`) lavora sugli attributi di classe e non sui singoli oggetti.
- `cls` : come possiamo notare nell'esempio; il primo parametro convenzionalmente chiamato nel metodo di classe è il `cls` (che sta per **class**), invece del canonico `self` .

Questo perché il parametro `self` da un metodo di istanza mentre il parametro `cls` da un metodo di classe.

Sia il decoratore che il parametro `cls` vanno usati insieme:

se si omette `@classmethod` , Python interpreterà `cls` come `self` , causando degli errori.

Esempio senza `@classmethod` :



Python

```
1 class Persona:
2     # Attributo di classe (condiviso)
3     conteggio_personone = 0
4
5     def __init__(self, nome: str):
6         self.nome = nome # Attributo di istanza (unico per
7         self.aggiorna_conteggio()
8
9
10    # SENZA @classmethod (ERRATO)
11    def aggiorna_conteggio(cls): # cls viene trattato come self!
12        cls.conteggio_personone += 1 # Fallisce se chiamato dalla
13        classe
14
15    # Creazione di istanze
16    alice = Persona("Alice W.")
17    bob = Persona("Bob M.")
18
19    print(f"Persone create: {Persona.conteggio_personone}") # Output:
20    2
```

La differenza tra `cls` e `self` :

- `self` :
permette di accedere solo agli attributi legati all'istanza.
- `cls` :
permette di accedere solo agli attributi legati alla classe.
- Non è possibile accedere a un attributo definito con `self` usando `cls` :
se provi a farlo, otterrai un **errore** perché l'attributo è definito solo **nell'istanza** e non nella **classe**.



Python

```
1 class Example:
2     def __init__(self) -> None:
3         self.instance_attr = "I am an instance attribute"
4
5     @classmethod
6     def show_attr(cls) -> None:
7         print(cls.instance_attr) # Errore! instance_attr non
8                                   esiste a livello di classe
9
10    ex = Example()
11    ex.show_attr() #Output: AttributeError: type object 'Example' has
12                  no attribute 'instance_attr'
```

Metodi speciali

I **metodi speciali** (detti anche **magic methods** o **dunder methods**, da "double underscore") sono metodi che iniziano e finiscono con due underscore (`__nome__`).

Essi **permettono di modificare il comportamento predefinito degli oggetti in Python**, ad esempio come vengono stampati, come sono confrontati, o addirittura permettono di trattarli come funzioni.

Il metodo `__str__`

Il metodo `__str__` serve per:

fornire una rappresentazione leggibile di un oggetto sotto forma di stringa.
Quando chiamiamo `print()` su un oggetto, Python cerca di chiamare automaticamente il metodo `__str__` per ottenere la stringa da stampare.

Esempio:



Python

```
1 class Person:
2     def __init__(self, name: str, age: int) -> None:
3         self.name = name
4         self.age = age
5
6     def __str__(self) -> str:
7         return f"{self.name}, {self.age} years old"
8
9 p = Person("Luca", 30)
10 print(p) # Output: Luca, 30 years old
```

Senza `__str__`

Se non definissimo il metodo `__str__`, Python stamperebbe una **rappresentazione predefinita** dell'oggetto, ad esempio:

C#

C#

```
1 <__main__.Person object at 0x7f5a6c8b2d90>
```

Ovvero l'indirizzo di memoria dell'oggetto, poco utile e poco leggibile.

✓ Perché è utile `__str__` ?

- **Migliora la leggibilità:** rende immediato capire il contenuto dell'oggetto.
- **Facilita il debug:** è più semplice controllare lo stato degli oggetti stampandoli.
- **Comodità:** non serve creare un metodo separato (es. `printMsg()`) da chiamare manualmente; basta `print(object)`.

☰ Se avessimo definito invece un metodo tipo `printMsg()`, per stampare il contenuto avremmo dovuto scrivere `print(p.printMsg())`. Con `__str__`, basta semplicemente `print(p)`.

Metodo `__call__`

Il metodo `__call__` :

permette di chiamare un oggetto come se fosse una funzione .

In pratica, aggiunge il comportamento delle **parentesi** `()` agli oggetti.



Python

```
1 class Greeter:
2     def __init__(self, message: str) -> None:
3         self.msg = message
4
5     def __call__(self) -> str:
6         return f"Hello, {self.msg}"
7
8 g = Greeter("Alice")
9 print(g()) # Output: Hello, Alice
```

In questo caso:

- `g()` esegue il metodo `__call__` , **anche se** `g` **è un oggetto**, non una funzione vera e propria.
- L'oggetto diventa **funzionale**: può essere "chiamato"

✓ Quando è utile `__call__` ?

- Quando vuoi **modellare un oggetto che si comporta come una funzione**.
- Quando desideri **salvare uno stato interno** (attributi) e allo stesso tempo permettere la chiamata diretta con `()` .

Il metodo `__new__(cls[, ...])`

Viene chiamato prima di `__init__()` , ed è usato per creare l'oggetto .

Serve soprattutto quando:

- **Si estende classi immutabili** (`int` , `str` , `tuple` , etc.)
- **Si ha la necessità di controllare o personalizzare l'istanza prima della creazione.**

Infatti questo metodo viene chiamato per creare una **nuova istanza** della classe (`cls`).

Questo **metodo è statico**, difatti Python lo gestisce in modo speciale, quindi non serve dichiararlo come tale.

È il metodo statico della classe che crea l'oggetto e restituisce un'istanza della classe appena creata, di default la classe `__new__()` invoca l'istanza della classe invocando il `new` della superclasse.

La superclasse di ogni Classe in Python è la classe `Object`, ed è il motivo per cui il metodo `__new__()` funziona: ogni volta che si invoca la superclasse di una classe che non specifica apertamente la superclasse Python riporta che l'ereditarietà arriva dalla classe `Object`.



Python

```
1  from beartype import beartype
2  from typing import Self
3  class Persona(object):
4      nome:str
5      cognome:str
6      def __new__(cls)->Self:
7          return super().__new__(cls)
8      def __init__(self, name)->None:
9          self.nome = nome
10     def fun(self, cognome):
11         self.cognome
12
13  if __name__ == '__main__':
14     persona = Persona("Alice")
15     print(type(Persona))
16     print("Superclasse di Persona:" + str(Persona.mro()[-1]))
```

`mro`: è l'ordine di risoluzione dei metodi che cerca prima quel metodo o quei metodi nelle sottoclassi, per poi risalire la gerarchia fino alla superclasse.

Una buona pratica è quella di definire gli attributi fuori dall' `__init__()` e poi inizializzarli nell' `__init__()`.

Questo perché se si avesse una classe con 10 attributi vengono definiti prima e quindi il codice sarebbe più ordinato e leggibile.

BearType: cosa fa? se lancio questo modulo annotando la funzione `beartype` mi controlla i tipi.

Sintassi del metodo `__new__(cls[, ...])`

Come primo argomento prende la classe da cui si vuole creare l'istanza (`cls`), e gli altri argomenti sono quelli che vengono passati nel momento in cui si invoca il costruttore (cioè quando si scrive `Classe(...)`).

Questo metodo restituisce una nuova istanza dell'oggetto (di solito è un'istanza `cls`).

L'implementazione tipica di questo metodo è la seguente:

Supponiamo di voler creare una sottoclasse di `str` che forza tutto in maiuscolo, e lo facciamo tramite il metodo `__new__()`.



Python

```
1 class UpperStr(str):
2     def __new__(cls, value):
3         print(f"__new__ ricevuto: {value}")
4         # Creiamo una nuova istanza usando la superclasse str
5         instance = super().__new__(cls, value.upper()) # <--
6         print(f"__new__ restituisce: {instance}")
7         return instance
8
9     def __init__(self, value):
10        print(f"__init__ chiamato con: {value}")
11
12
13 s = UpperStr("ciao")
14 print(s)           # Output: CIAO
15 print(type(s))     # <class '__main__.UpperStr'>
```

Spiegazione:

- `super().__new__(cls, value.upper())` :
chiama `str.__new__()` passando il valore già modificato.
- `UpperStr("ciao")` :
creerà un oggetto `str` con valore `"CIAO"` e lo restituirà.
- `__init__()` :
viene comunque chiamato dopo con il valore originale `"ciao"`.

Come detto prima questo approccio è necessario con classi immutabili (`str`, `int`, `tuple`, etc.), perché `__init__()` non può modificarle (ma solo inizializzare i valori degli attributi), di conseguenza ogni modifica deve essere fatta in `__new__()`.

Se `__new__()` restituisce un'istanza di `cls` (cioè della stessa classe per cui è stato chiamato il costruttore), allora `__init__()` verrà automaticamente invocato

per inizializzarla.

Se invece `__new__()` restituisce un oggetto di un tipo diverso, allora `__init__()` **non verrà chiamato**, poiché Python presume che l'oggetto sia già pronto all'uso.

Esempio pratico:



Python

```
1 class MyInt(int):
2     def __new__(cls, value):
3         print("Sto creando un MyInt con valore", value)
4         # Raddoppia sempre il valore dato
5         return super().__new__(cls, value * 2)
6
7 a = MyInt(3)
8 print(a) # Output: 6
```

Nota

`__new__()` deve sempre restituire l'istanza solitamente chiamando `super().__new__(cls, ...)`

In sintesi: `__new__(cls[, ...])`

- È usato comunemente per creare istanze di tipi immutabili (come ad esempio `int`, `str`, `tuple`), al fine di personalizzare la creazione.
- Viene anche sovrascritto nei metaclassi per personalizzare la creazione delle classi

Differenza tra il metodo `__new__()` e il metodo `__init__()`

1. `__new__(cls, ...)`:
 - È responsabile della creazione vera e propria dell'oggetto.
 - Riceve `cls`, cioè la classe da cui si vuole creare l'oggetto.
 - Ritorna l'oggetto creato (tipicamente si scrive `return super().__new__(cls)`)
 - È utile per classi immutabili, dove non puoi modificare l'oggetto dopo la creazione (es: `int`, `str`, `tuple`)
2. `__init__(self, ...)`:

- viene chiamato dopo che l'oggetto è stato creato da `__new__()` .
- Inizializza l'oggetto appena creato, impostando i suoi attributi.
- Agisce sull'istanza, quindi usa `self`

✓ In altre parole:

◆ `__new__()` :

crea l'oggetto, è usato soprattutto per controllare la creazione (es. classi immutabili).

◆ `__init__()` :

inizializza l'oggetto creato, settando gli attributi dell'istanza.

☰ In sintesi

- `__new__()`

Viene chiamato prima di `__init__()` e serve a creare l'oggetto della classe.

È particolarmente utile quando si lavora con **classi immutabili** (`str` , `int` , `tuple` , ecc.), **perché permette di modificare i dati prima della creazione vera e propria dell'oggetto.**

Restituisce l'istanza appena creata (di solito usando `super().__new__(cls, ...)`).

- `__init__()`

Viene chiamato dopo che l'oggetto è stato creato, e serve a inizializzare l'istanza, cioè a settare gli attributi tramite `self` .

Non può modificare l'oggetto già creato, solo inizializzarlo.

Il metodo `__eq__(self, other)`

Questo metodo appartiene ai cosiddetti metodi di confronto "ricchi" o rich comparison (`__ne__` , `__gt__` , `__ge__` , etc.).

Per intenderci i normali **operatori di confronto** (`==` , `!=` , `<` , `<=` , `>` , `>=`) corrispondono a metodi speciali:

Operatore	Metodo Chiamato
<code>x < y</code>	<code>x.__lt__(y)</code>
<code>x <= y</code>	<code>x.__le__(y)</code>
<code>x == y</code>	<code>x.__eq__(y)</code>
<code>x != y</code>	<code>x.__ne__(y)</code>
<code>x > y</code>	<code>x.__gt__(y)</code>
<code>x >= y</code>	<code>x.__ge__(y)</code>

- Questi metodi possono restituire:
 - `True` o `False` **per un confronto riuscito,**
 - **oppure lo speciale valore** `NotImplemented` **se il confronto non è supportato tra i due oggetti.**
- Se usati in un contesto booleano (es. `if x == y:`), Python applica automaticamente `bool()` al risultato
 Come possiamo vedere da questa tabella sopra il metodo `__eq__()` confronta l'identità (cioè `x is y`) e restituisce `True` solo se sono lo stesso **oggetto in memoria.**

A cosa serve `__eq__(self, other)`

Definisce quando due oggetti sono uguali (`==`).

Se **non** viene definito, Python utilizza il comportamento predefinito della classe base `object`, che è equivalente a `x is y` (cioè verifica se sono lo stesso oggetto in memoria).

Ma se si vuole confrontare i **contenuti** degli oggetti, puoi ridefinire `__eq__()` per specificare tu cosa significa "uguale".

```

1  from beartype import beartype
2  from typing import Self
3  class Persona(object):
4      nome:str
5      cognome:str
6      def __new__(cls)->Self:
7          return super().__new__(cls)
8      def __init__(self,name)->None:
9          self.nome = nome
10     def fun(self, cognome):

```

```

11         self.cognome
12
13     if __name__ == '__main__':
14         alice1: Persona = Persona("Alice")
15         print(type(alice1))
16         print("Superclasse di Persona:" + str(Persona.mro()[-1]))
17         alice2:Persona = Persona ("Alice")

```

In questo modo restituisce false se dico che alice1 e alice2 sono la stessa Persona:



Python

```

1  from beartype import beartype
2  from typing import Self, Any
3  class Persona(object):
4      nome:str
5      cognome:str
6      def __new__(cls)->Self:
7          return super().__new__(cls)
8      def __init__(self,name)->None:
9          self.nome = nome
10     def __eq__(self, other:Any):
11         if not isinstance(other, Persona ): #o type(self) al
posto di Persona
12             return False
13         return self.nome ==other.name
14
15     if __name__ == '__main__':
16         alice1: Persona = Persona("Alice")
17         print(type(alice1))
18         print("Superclasse di Persona:" + str(Persona.mro()[-1]))
19         alice2:Persona = Persona ("Alice")

```

In questo caso due persone sono la stessa Persona, questa cosa non è sempre vera, dipende da come gestiamo l'uguaglianza:

```

1  from beartype import beartype
2  from typing import Self, Any
3  class Persona(object):
4      nome:str
5      cognome:str
6      def __new__(cls)->Self:

```

```

7         return super().__new__(cls)
8     def __init__(self, name) -> None:
9         self.nome = name
10    def __eq__(self, other: Any):
11        if not isinstance(other, Persona ):  #o type(self) al
posto di Persona
12            return False
13        return self.nome == other.name
14
15    if __name__ == '__main__':
16        alice1: Persona = Persona("Alice")
17        print(type(alice1))
18        print("Superclasse di Persona:" + str(Persona.mro()[-1]))
19        alice2: Persona = Persona ("Alice")
20
21    bob: Persona = Persona("bob")
22
23    l: list[Persona] = ["alice1", "bob", "alice2"]
24    l: list[Persona] = ["alice1", "bob"]
25    alice2 in l

```

Uscira true perchè cerca ogni istanza di alice quindi definisce l'uguaglianza tra i due elementi delle due liste alice1.

Ovviamente non posso farlo con i dizionari perchè le chiavi sono hashabili.

Esempio pratico



Python

```

1  class Persona:
2      def __init__(self, nome, eta):
3          self.nome = nome
4          self.eta = eta
5
6      def __eq__(self, other):
7          if isinstance(other, Persona):
8              return (self.nome == other.nome) and (self.eta ==
other.eta)
9              return NotImplemented
10
11  a = Persona("Mario", 30)
12  b = Persona("Mario", 30)
13  c = Persona("Luigi", 25)
14

```

```
15 print(a == b) # True
16 print(a == c) # False
```

Sintassi di `__eq__(self, other)`

1. `self`:

- È il riferimento all'istanza corrente della classe (quella a sinistra dell'operatore `==`).
- Permette di accedere agli attributi dell'oggetto creati con `__init__`.

esempio:



Python

```
1 a = Persona("Mario", 30)
```

In questo esempio `a == b`; il `self` sarà l'oggetto `a`.

2. `other`:

Questo parametro va ad indicare l'oggetto con cui si confronta `self`.

Quindi è quello a destra del `==`

Può essere oggetto della stessa classe oppure **di un'altra classe o tipo**.

Nel metodo `__eq__`, si controlla quasi sempre prima se `other` è della classe usando `isinstance()`:



Python

```
1 def __eq__(self, other):
2     if isinstance(other, Persona):
3         return self.nome == other.nome and self.eta == other.eta
4     return NotImplemented
```

In questo esempio:

- `self` è l'oggetto `a`
- `other` è l'oggetto `b` nel confronto `a == b`

👁 **Perché serve `other`?**

Serve per **prendere i suoi attributi** e confrontarli con quelli di `self`.

È come dire: “Se `other` è un'altra Persona, vediamo se ha lo stesso nome e la stessa età di me (`self`).”

Visualizzazione

Nel confronto `a == b`, Python esegue internamente:



Python

```
1 a.__eq__(b) # self = a, other = b
```

Io in questo caso posso personalizzare l'istanza ad esempio se due Persone hanno lo stesso codice fiscale io posso fare in modo che siano la stessa Persona.

La funzione `__hash__(self)`

Una funzione hash prende cose all'interno di un dominio qualsiasi e restituisce un codice.

In python una funzione hash prende cose e ritorna un intero.

La lista non è hashable perché una lista è mutabile.

L 'hash' è uguale per oggetti equivalenti, però può essere che ci siano cose diverse che hanno lo stesso hash, questa cosa si chiama collisione.

Quindi l'hash è limitato è finito, infatti ci sono stringhe diverse con lo stesso hash. Con numeri piccoli di hash posso avere più collisioni, con numeri grandi o meno possibilità di collisione.

Le funzioni hash sono dette anche one-way function: perché

`__hash__(self)`

- È il metodo speciale chiamato dalla funzione incorporata `hash(obj)` da operazioni su **collezioni indicizzate tramite hash** come `'set'`, `frozenset` e `'dict'`.
- Deve restituire un **intero**.

In sostanza: **serve per rendere un oggetto utilizzabile, come chiave nei dizionari o nei set, calcolando un valore numerico; ovvero l'hash** (per vedere cos'è un hash vedi la lezione sulle [Collections](#), in particolare i capitoli [Hash](#) e [Gli hasher nei dizionari](#)).

Infatti il requisito fondamentale per questo metodo è:

Se due oggetti sono uguali (`x == y` restituisce `True`), devono avere lo stesso valore di hash .

Hash prende un singolo parametro.

🔗 Per oggetti personalizzati, puoi costruire l'hash combinando i valori dei componenti usati anche per l'uguaglianza:



Python

```
1 def __hash__(self):
2     return hash((self.name, self.nick, self.color))
3
```

Infatti in questo caso si sta mettendo tre valori dentro una tupla perchè così è come se si stesse mettendo un solo valore

🔗 **Maggiori informazioni sul metodo** `__hash__()` >

- `hash()` tronca il valore secondo l'architettura (64 bit o 32 bit).
- Per garantire compatibilità tra diverse piattaforme, verifica la larghezza con:

```
1 python -c "import sys; print(sys.hash_info.width)"
```

Regole sull'implementazione:

- Se una classe non definisce `__eq__()` , non dovrebbe definire `__hash__()` !
Se la classe a sia `__hash__()` che `__eq__()` vuol dire che l'oggetto è immutabile
- Se una classe definisce `__eq__()` ma non `__hash__()` , Python disabilita automaticamente l'hash: **le istanze non potranno essere usate come chiavi nei dizionari o elementi nei set** .
- Se una classe è mutabile e definisce `__eq__()` , non deve definire `__hash__()` , per evitare comportamenti incoerenti nelle collezioni hashabili .

⚠ **Se si definisce `__eq__()`, si deve anche definire `__hash__()` per mantenere la coerenza: oggetti considerati uguali devono avere lo stesso hash.**

Una collezione hashable è una collezione che usa l'hash, i dizionari usa l'hash di quella chiave (viene detto hash map).

Esistono librerie che calcolano l'hash, perché ogni volta che lanciamo python l'hash sarà diverso mentre con delle librerie esterne il numero di hash sarà sempre lo stesso.

Quindi non devo usare il built-in hash di Python, meglio usare `hashlib`, perché questa libreria restituisce un codice hash sempre uguale.

```
1  from beartype import beartype
2  from typing import Self, Any
3  class Persona(object):
4      nome:str
5      cf:str
6      def __new__(cls)->Self:
7          return super().__new__(cls)
8      def __init__(self,name)->None:
9          self.nome = nome
10     def __eq__(self, other:Any):
11         if not isinstance(other, Persona ):  #o type(self) al
        posto di Persona
12             return False
13         return self.cf ==other.cf
14
15     def __hash__(self):
16         return hash(self.cf)
17     def __repr__(self):
18         return f"Persona {self.cf} e nome: {self.nome}"
19
20 if __name__ == '__main__':
21     alice1: Persona = Persona("Alice")
22     print(type(alice1))
23     print("Superclasse di Persona:" + str(Persona.mro()[-1]))
24     alice2:Persona = Persona (nome:"Alice", cf: "AA")
25
26     bob:Persona = Persona("bob")
27
28     l:list[Persona] = ["alice1", "bob", "alice2"]
29     l: list[Persona] = ["alice1", "bob"]
```

Esempi pratici del metodo `__hash__(self)`



Python

```

1  class Cat:
2      def __init__(self, name, color):
3          self.name = name
4          self.color = color
5
6      def __eq__(self, other):
7          if not isinstance(other, Cat):
8              return NotImplemented
9          return self.name == other.name and self.color ==
other.color
10
11     def __hash__(self):
12         return hash((self.name, self.color))  # Combinazione
immutabile
13
14  c1 = Cat("Milo", "black")
15  c2 = Cat("Milo", "black")
16  c3 = Cat("Luna", "white")
17
18  print(c1 == c2)  # True
19  print(hash(c1), hash(c2))  # Uguali
20  print({c1, c2, c3})  # Solo 2 oggetti diversi nel set

```

Spiegazione:

- `__eq__()` confronta due oggetti `Cat` in base a **name e color**.
- `__hash__()` calcola un hash basato su una **tupla immutabile** `(self.name, self.color)`. Usare una tupla è una buona pratica perché:
 - Le tuple sono immutabili $\Rightarrow$ l'hash non cambia nel tempo.
 - Il risultato è coerente con `__eq__()`: due oggetti uguali $\rightarrow$ stesso hash.

Esempio problematico



Python

```

1  class Dog:
2      def __init__(self, name):

```

```

3         self.name = name
4
5     def __eq__(self, other):
6         return self.name == other.name
7
8     def __hash__(self):
9         return hash(self.name) # ⚠ Potenziale problema se
    'name' cambia
10
11 d = Dog("Rex")
12 s = {d}
13 d.name = "Fido"
14 print(d in s) # ⚠ False: l'hash è cambiato! Rende il set
    inconsistente

```

Spiegazione:

- `__eq__()` confronta solo il `name`.
- `__hash__()` usa `self.name`, che è **modificabile** dopo la creazione dell'oggetto.
- Questo può portare a **comportamenti incoerenti** nelle collezioni hash-based.
- Quando `d` è stato aggiunto al set, l'hash era basato su `"Rex"`.
- Dopo aver cambiato `name`, il valore dell'hash cambia.
- Quando il set prova a cercare `d`, calcola un nuovo hash (per `"Fido"`) → non trova l'oggetto → `in` restituisce `False`.

Esempio pratico: Python disabilita automaticamente `__hash__()` se definisci solo `__eq__()`



Python

```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def __eq__(self, other):
6         return isinstance(other, Person) and self.name ==
    other.name
7
8 p1 = Person("Alice")
9 p2 = Person("Alice")
10

```

```
11 print(p1 == p2) # True: __eq__ funziona
12 print(hash(p1)) # TypeError!
```

Spiegazione:

La classe `Person` ha `__eq__()`, ma non ha `__hash__()`.

L'errore in output viene generato perché

- Python presume che se stai confrontando oggetti con `__eq__()`, allora vuoi che il confronto abbia un significato logico (non solo l'identità).
- Ma se non definisci anche `__hash__()`, Python protegge la coerenza tra `==` e `hash()` disabilitandolo del tutto:

"Oggetti uguali devono avere lo stesso hash. Se non puoi garantirlo, non ti lascio usare hash."

Quindi `__hash__()` viene definito dal programmatore stesso quando si è sicuri che l'oggetto sia immutabile o che l'hash sia stabile:



Python

```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def __eq__(self, other):
6         return isinstance(other, Person) and self.name ==
other.name
7
8     def __hash__(self):
9         return hash(self.name)
10
11 p1 = Person("Alice")
12 print(hash(p1)) # OK!
```

VEDI PER IL METODO `__repr__()`